Metadata-Driven Memory Control for Long-Horizon AI Agents

Version 1.0 — 2026-08-12
Author: Marco Rapp
Repository: `MarcoRapp/whitepaper-labs`

Abstract

Long-horizon AI agents increasingly rely on persistent memory systems to maintain continuity across extended tasks, sessions, and environments. As memory stores grow, however, the cost of repeatedly searching, re-evaluating, and semantically reasoning over the full memory space can become substantial. At the same time, purely reactive retrieval can fail when important information is old, weakly similar to the current query, or no longer salient in the active context.

This paper specifies and positions a **metadata-driven memory control architecture** in which an AI-based memory manager performs semantic understanding primarily when memory state changes, then stores the result as a compact control layer composed of tags, weights, persistence rules, triggers, polling schedules, and lifecycle metadata. The memory manager can subsequently operate primarily on this control layer and retrieve full memories only when activation conditions are met.

The central hypothesis is:

> **Semantic understanding should be paid for when memory state changes, not repeatedly whenever memory might become relevant.**

The proposed architecture combines AI-based memory lifecycle management with weighted metadata, proactive polling, event-triggered activation, conflict detection, selective retrieval, and explicit persistence policies. It is intended as a control layer that can operate alongside vector stores, graph memories, episodic memory systems, and existing memory managers rather than replacing them.

The design extends concepts introduced in the earlier Whitepaper Labs work on **memory architecture** and **polling-based flagging for contextual relevance**.

---

1. Problem Statement

Long-running AI agents accumulate large amounts of information:

- user preferences,

- contractual obligations,
- deadlines,
- task states,
- environmental observations,
- prior hypotheses,
- financial transactions,
- tool outputs,
- conversation history,
- completed actions,
- recurring obligations,
- and temporary operational details.

Not all of this information is equally important at all times.

A common strategy is to store memories and retrieve them when the current query appears semantically related. This works well for many short-horizon interactions, but it introduces two fundamental problems in long-horizon systems.

1.1 Relevance decay in long trajectories

An important fact may remain true even when it is no longer recent.

Example:

> The vending location charges a recurring daily fee.

Hundreds or thousands of steps later, the agent observes a small debit from its account. If the original obligation is no longer salient and is not retrieved, the agent may interpret the transaction as unauthorized.

The problem is not necessarily lack of intelligence. The system may simply fail to reactivate the correct piece of old state.

1.2 Repeated semantic evaluation is expensive

If a system has already determined that a memory represents:

- a recurring obligation,
- a permanent user preference,
- a deadline,
- a legal constraint,
- a completed task,
- or a temporary observation,

then repeatedly asking a large reasoning model to rediscover that classification is wasteful.

A memory manager that continuously reasons over the complete memory store may therefore scale poorly as the memory space grows.

The proposed alternative is to separate:

1. **semantic understanding of memories**, and
2. **ongoing relevance control**.

---

2. Relation to Previous Whitepaper Labs Work

This architecture builds on two earlier concepts in the Whitepaper Labs project.

2.1 Memory Management and Layering

The earlier memory architecture work proposed structured separation of persistent information rather than treating memory as a single undifferentiated store.

The present paper retains that principle while introducing a separate **control plane** above the memory content itself.

2.2 Polling-Based Flagging System

The polling-based flagging concept proposed that important contextual information should not depend exclusively on passive retrieval. Instead, relevant information can be flagged and periodically re-evaluated so that important state is not lost merely because it has moved far back in the context.

The present architecture generalizes that idea into a broader AI-managed memory lifecycle.

Polling becomes one of several activation mechanisms alongside event triggers, conflict triggers, persistence rules, and weighted metadata.

---

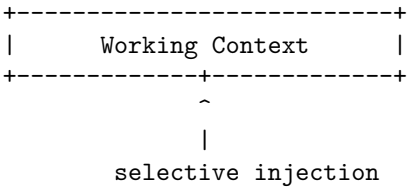
3. Core Design Principle

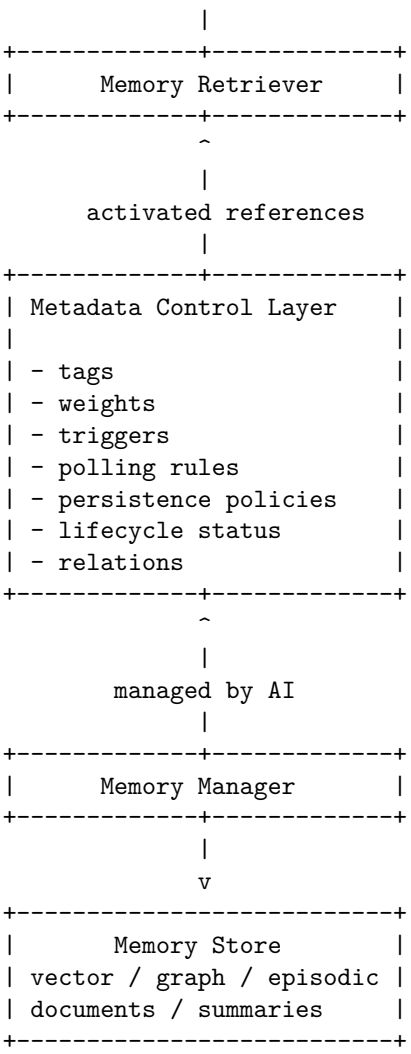
The architecture is based on a simple separation:

> **Manage relevance first. Retrieve memory second.**

Instead of repeatedly searching and reasoning across the full memory store, the system maintains a smaller metadata control layer.

The memory manager primarily operates on this compact structure and accesses full memory content only when necessary.





The architecture does not require a specific memory backend.

The metadata control layer can reference:

- vector database entries,
- graph nodes,
- episodic memories,
- summaries,
- documents,
- structured state,
- or other persistent representations.

---

4. AI-Based Memory Manager

The memory manager may itself be AI-based.

This is useful because memory classification often requires semantic understanding rather than simple keyword matching.

When new information arrives, the manager can decide whether to:

- create a new memory,
- update an existing memory,
- merge related memories,
- ignore irrelevant information,
- archive completed information,
- soft-delete obsolete state,
- hard-delete information where policy permits,
- create or reuse tags,
- assign weights,
- assign triggers,
- define polling behavior,
- establish persistence rules,
- detect relationships,
- or mark information as conflicting or superseded.

4.1 Semantic phase

The expensive semantic phase occurs when the memory state changes.

```
New information
      |
      v
AI Memory Manager
      |
      +--> understand
      +--> classify
      +--> tag
      +--> assign priority
      +--> assign persistence
      +--> assign triggers
      +--> define polling
      +--> merge/update/create
      |
      v
Memory Store + Metadata Layer
```

4.2 Operational phase

Once classification has been performed, the system should avoid paying for the same semantic reasoning repeatedly.

The normal operational loop can work primarily on metadata.

```
Current event
    |
    v
Cheap metadata evaluation
    |
    v
Activated tags / flags
    |
    v
Small candidate memory set
    |
    v
Selective retrieval
    |
    v
Main reasoning model
```

This leads to the core efficiency principle:

> **Use AI where semantic understanding is required, but do not repeatedly pay for semantic understanding that has already been performed.**

---

5. Metadata as a Control Plane

Tags should not be treated as simple labels attached to individual memories.

Instead, they should be first-class control objects.

A tag can reference many memories, and a memory can belong to many tags.

Example:

```
financial_obligation [weight: 0.95]
 +- office rent agreement
 +- supplier payment terms
 +- insurance premium
 +- electricity contract

inventory [weight: 0.72]
 +- supplier payment terms
 +- pending shipment
 +- current stock state
```

This creates a reusable semantic index over the memory space.

5.1 Example tag structure

```yaml
tag: recurring_financial_obligation
weight: 0.95
persistence: hard
event_triggers:
  - financial_transaction
  - invoice_received
poll_interval: 24h
decay: false
references:
  - memory_184
  - memory_912
  - memory_1402
```

The memory manager can modify the metadata without repeatedly parsing the full referenced memories.

The control record may also carry **provenance, version, validity interval, and supersession metadata**. These fields are not claimed as novel: temporal memory systems such as Zep/Graphiti and operating-system-style proposals such as MemOS already make temporal validity, provenance, or versioning explicit. Their role here is narrower: they give the control plane cheap signals for conflict checks and routing before full content is retrieved.

---

6. Relevance, Priority, and Persistence Are Different

A core requirement is to separate three concepts that are often conflated.

6.1 Relevance

How useful is the information to the current situation?

6.2 Priority

How important is it that the system does not miss or mishandle this information?

6.3 Persistence

How long should the information remain available?

These values are not interchangeable.

Example:

```
Memory:
"Office rent is €1,500 per month."
```

```
Current relevance: 0.10
Priority:          0.95
Persistence:       HARD
```

The rent may not matter to the current conversation, but it must not decay simply because it is temporarily irrelevant.

By contrast:

```
Memory:
"Pay invoice #384 before August 14."

Current relevance: 0.98
Priority:          0.98
Persistence:       UNTIL_COMPLETED
```

After payment, the active memory can be archived or removed from the working relevance system.

---

7. Persistence Policies

The system should support explicit persistence classes.

Possible classes include:

HARD

The memory remains persistent until explicitly changed.

Examples:

- contracts,
- recurring obligations,
- critical user constraints,
- safety requirements.

UNTIL_COMPLETED

The memory remains active until a completion condition is detected.

Examples:

- invoices,
- tasks,
- appointments,
- one-time obligations.

SOFT

The memory may decay based on relevance, age, or reinforcement.

Examples:

- preferences,
- working assumptions,
- low-value contextual details.

EPHEMERAL

The memory is expected to expire quickly.

Examples:

- temporary UI state,
- short-lived tool output,
- transient execution details.

ARCHIVE

The memory is no longer actively relevant but remains available for historical reconstruction.

This is preferable to immediate hard deletion in many cases.

---

8. Hybrid Relevance Scheduling

The architecture combines multiple mechanisms for reactivating memories.

8.1 Event-triggered activation

An event can activate relevant tags.

```
Bank transaction observed
        |
        v
financial_event
        |
        v
activate:
- recurring_financial_obligation
- supplier_cost
- subscription_fee
        |
        v
retrieve associated memories
```

This avoids broad retrieval.

8.2 Proactive polling

Some information should become salient even when the current context does not naturally request it.

Example:

```
Tag: rent
poll_interval: monthly
```

Polling can ask:

- Is this obligation still active?
- Has it changed?
- Is a payment due?
- Has the completion condition been satisfied?
- Should the weight increase or decrease?

This solves a limitation of purely reactive retrieval:

> To retrieve forgotten information, the system often needs enough context to know what it should retrieve.

Polling allows important state to reactivate itself.

This requirement is closely related to **prospective memory**: retaining an intention or obligation while other activity continues, then executing or surfacing it when a future time, event, or state cue occurs. PM-Bench (Liu and Gabriel, 2026) shows that this remains difficult for contemporary LLM agents even under controlled conditions. The present proposal does not claim prospective memory as a new capability; rather, event triggers and polling are proposed as control-plane mechanisms that can implement time-based and event-based prospective-memory behavior for persistent agent state.

8.3 Conflict triggers

New observations should be checked against high-priority persistent state before being promoted into the agent's world model.

Example:

```
Known:
Daily location fee = $2

Observed:
-$2 transaction

Initial hypothesis:
"Unauthorized withdrawal"
```

```
Conflict detection:
financial_obligation tag = HIGH

Retrieve:
location agreement

Result:
transaction is expected
hypothesis rejected
```

This can reduce self-reinforcing error chains in long-running agents.

---

9. Memory Lifecycle Management

The AI memory manager should support a full lifecycle.

```
CREATE
   |
CLASSIFY
   |
TAG
   |
WEIGHT
   |
ACTIVE
   |
   +--> UPDATE
   +--> MERGE
   +--> REWEIGHT
   +--> POLL
   +--> RETRIEVE
   |
COMPLETION / OBSOLESCENCE
   |
   +--> ARCHIVE
   +--> SOFT DELETE
   +--> HARD DELETE
```

Deletion and destructive rewriting should be conservative. Recent evidence indicates that repeated LLM-based consolidation can degrade useful memories even when the underlying experiences are sound (Zhang et al., 2026). The metadata control layer should therefore preserve provenance to source memories and prefer reversible lifecycle transitions over destructive replacement when practical.

A memory manager can make mistakes, so systems should distinguish between:

- active removal from retrieval,
- archival,
- reversible soft deletion,
- and irreversible deletion.

---

10. Resource Efficiency Hypothesis

The key performance claim of this paper is intentionally stated as a hypothesis rather than an established result.

Hypothesis H1

A metadata-first control layer can reduce the amount of semantic reasoning required during long-horizon memory management.

Hypothesis H2

A memory manager operating primarily on tags, weights, triggers, and lifecycle metadata can reduce retrieval search space compared with repeatedly evaluating the full memory store.

Hypothesis H3

Proactive polling and persistent control metadata can improve recall of old but objectively important information that would otherwise be missed by purely similarity-based retrieval.

Hypothesis H4

The active metadata space can grow more slowly than the underlying memory store if tags are reused, merged, generalized, and pruned.

A hypothetical system might contain:

```
100,000 memories
800 active semantic tags
50 triggered tags in the current domain
12 candidate memories
5 memories injected into context
```

The memory manager therefore does not need to reason over all 100,000 memories during every step.

---

11. Preventing Tag Explosion

Metadata-first management fails if every memory creates a unique tag.

The AI manager must therefore be able to:

- reuse existing tags,
- merge synonymous tags,
- form hierarchical relationships,
- generalize overly specific tags,
- decay unused tags,
- archive obsolete tags,
- and prevent duplicate semantic categories.

Example:

```
office_rent
building_rent
monthly_workspace_fee
location_payment
```

may be consolidated into:

```
recurring_location_cost
```

with child relations where necessary.

The control layer must remain substantially smaller and cheaper to manage than the memory store it controls.

---

12. Multi-Tier Compute Architecture

The architecture can separate cheap control logic from expensive reasoning.

```
Tier 1: Deterministic / lightweight
- timers
- event matching
- simple weight changes
- deadline checks

        |
        v

Tier 2: Small AI memory manager
- classification
- tag selection
- lifecycle decisions
- merge decisions
- candidate ranking
```

```
        |
        v

Tier 3: Retrieval layer
- vector search
- graph traversal
- document fetch
- episodic memory access

        |
        v

Tier 4: Large reasoning model
- complex task reasoning
- planning
- decision-making
```

The system can therefore reserve high-cost reasoning for cases where it provides meaningful value.

---

13. Compatibility With Existing Memory Systems

This architecture is not proposed as a replacement for:

- vector databases,
- semantic retrieval,
- graph memory,
- episodic memory,
- long-context models,
- or existing AI memory managers.

Instead, it acts as a control layer that reduces how often and how broadly those systems need to be queried.

> **The proposed architecture is a memory control plane, not a memory backend.**

Existing retrieval systems remain responsible for finding the actual content once the control layer has identified what deserves attention.

---

14. Evaluation Proposal

The architecture should be tested empirically against several baselines.

Baseline A — Large context only

Relevant history remains in the context window without dedicated memory control.

Baseline B — Similarity retrieval

Memories are retrieved primarily through vector similarity.

Baseline C — AI memory manager

A manager performs direct memory lifecycle operations over the memory store.

Experimental System — Metadata-Controlled AI Memory Manager

The proposed manager uses:

- weighted tags,
- persistence classes,
- polling,
- event triggers,
- conflict triggers,
- selective retrieval,
- and lifecycle management.

Metrics

Potential evaluation metrics include:

- task success over long horizons,
- critical-memory recall,
- false retrieval rate,
- missed-obligation rate,
- memory-manager tokens per agent step,
- number of full-memory evaluations,
- retrieval operations per step,
- memory-management latency,
- compute cost,
- tag count vs. memory count,
- incorrect deletion rate,
- stale-memory rate,
- recovery from false hypotheses,
- update fidelity under repeated revisions,
- interference resistance under distractor accumulation,
- preservation of raw/source evidence after consolidation,
- and temporal-validity/versioning accuracy.

---

15. Vending-Bench-Style Failure Scenario

A useful benchmark should contain important state that remains valid while becoming progressively less recent.

Example:

```
Step 1:
Agent learns that the vending location charges a recurring fee.

Step 300:
Agent performs inventory management.

Step 800:
Agent processes customer requests.

Step 1500:
Agent observes the recurring fee transaction.
```

A purely reactive system may fail to retrieve the original obligation.

A metadata-controlled system can preserve:

```
tag: recurring_location_cost
persistence: hard
event_trigger: financial_transaction
```

When the transaction appears, the relevant memory is reactivated before the system constructs a false fraud hypothesis.

This test directly measures whether persistent objective state survives long-horizon contextual drift.

---

16. Failure Modes and Limitations

The proposed architecture introduces its own risks.

16.1 Incorrect initial classification

If the AI manager assigns the wrong tags or persistence policy, later retrieval may fail.

16.2 Incorrect deletion

A manager may conclude that a memory is completed or obsolete when it is still needed.

Archival and reversible deletion should therefore be preferred where possible.

16.3 Tag explosion

Too many highly specific tags can eliminate the efficiency advantage.

16.4 Weight drift

Repeated interactions may accidentally increase the importance of a wrong hypothesis.

Conflict checks and provenance should therefore be included.

16.5 Manager hallucination

An AI-based memory manager can itself produce incorrect classifications.

Structured validation and deterministic rules should be used for critical state where possible.

16.6 Polling overhead

Polling every tag too frequently would recreate the scaling problem.

Polling frequency must therefore be selective and weighted.

16.7 Hidden dependencies

Some relevant memories may not share obvious tags with a current event.

The system must retain fallback semantic retrieval for cases where metadata routing fails.

---

17. Future Work

Future research should investigate:

- learned tag-weight adjustment,
- automatic tag hierarchy formation,
- provenance-aware conflict resolution,
- adaptive polling intervals,
- cross-agent shared metadata layers,
- privacy-aware persistence policies,
- hard constraints for legal and safety state,
- benchmark suites for long-horizon obligation retention,
- and hybrid systems combining graph memory with metadata-driven control.

A particularly important research direction is determining whether the metadata layer can remain sublinear relative to total memory growth in realistic long-running agents.

18. Conclusion

Long-horizon AI agents do not only need more memory.

They need better control over **when memory deserves attention**.

The proposed metadata-driven architecture separates expensive semantic understanding from ongoing relevance management. An AI-based memory manager classifies information when it enters or changes state, then maintains a compact control layer composed of tags, weights, persistence rules, triggers, and polling schedules.

This enables the system to reactivate important information without continuously reasoning over the complete memory store.

The central design principle is:

> **Understand once, represent structurally, manage cheaply, retrieve selectively, and re-evaluate only when necessary.**

If validated empirically, this architecture could reduce memory-management cost while improving retention of long-lived obligations and critical state in autonomous AI agents.

References and Prior Work

The proposed architecture sits within a rapidly developing body of work on agent memory, learned memory management, proactive memory intervention, structured indexing, and long-horizon evaluation. The following works are particularly relevant.

Internal foundation papers

- **Whitepaper Labs — Polling-Based Flagging System for Context Relevance in AI Memory Development.** Earlier project work motivating proactive re-evaluation of important contextual state rather than relying exclusively on passive retrieval.
- **Whitepaper Labs — Memory Management & Personality Layering in AI Systems.** Earlier project work motivating structured separation of persistent information and memory layers.

Foundational agent-memory architectures

- **Park et al. (2023), "Generative Agents: Interactive Simulacra of Human Behavior."** Generative Agents stores a natural-language

memory stream, synthesizes higher-level reflections, and dynamically retrieves memories for planning. Its retrieval design combines recency, importance, and relevance, establishing important prior art for weighted memory salience and selective recall. The present proposal therefore does not claim weighting or salience-based retrieval as novel; it instead separates relevance/priority/persistence and uses persistent metadata as an operational control layer. DOI: 10.1145/3586183.3606763. arXiv: 2304.03442. URL: https://arxiv.org/abs/2304.03442

- **Packer et al. (2023), "MemGPT: Towards LLMs as Operating Systems."** MemGPT applies operating-system-inspired virtual-context management to move information between constrained in-context memory and external storage. It is foundational prior art for tiered/managed agent memory. The present proposal differs by focusing on a backend-agnostic metadata control plane for activation, persistence, and routing rather than virtual-context paging itself. arXiv: 2310.08560. URL: https://arxiv.org/abs/2310.08560
- **Yao et al. (2023), "Cognitive Architectures for Language Agents (CoALA)."** CoALA provides a general cognitive architecture for language agents and distinguishes working, episodic, semantic, and procedural memory together with decision procedures over memory and action. This paper therefore treats memory-type layering as established background rather than a novelty claim. arXiv: 2309.02427. URL: https://arxiv.org/abs/2309.02427
- **Zhong et al. (2023), "MemoryBank: Enhancing Large Language Models with Long-Term Memory."** MemoryBank introduced long-term conversational memory with explicit forgetting/retention behavior inspired by the Ebbinghaus forgetting curve. It is relevant prior art for memory decay and reinforcement. The present architecture does not claim decay itself; it constrains decay through explicit persistence policy so that low current relevance does not automatically imply low retention priority. arXiv: 2305.10250. URL: https://arxiv.org/abs/2305.10250

Learned and structured memory management

- **Yan et al. (2026), "Memory-R1: Enhancing Large Language Model Agents to Manage and Utilize Memories via Reinforcement Learning," ACL 2026.** Memory-R1 uses a learned Memory Manager with explicit `ADD`, `UPDATE`, `DELETE`, and `NOOP` operations, together with a separate Answer Agent. This provides direct prior art for AI-based memory lifecycle management while differing from the present proposal's emphasis on a persistent metadata control plane and cheap ongoing relevance control. DOI: 10.18653/v1/2026.acl-long.583. URL: https://aclanthology.org/2026.acl-long.583/
- **Zhao et al. (2026), "Inside Out: Evolving User-Centric Core**

Memory Trees for Long-Term Personalized Dialogue Systems," ACL 2026. Inside Out maintains a structured `PersonaTree` and trains a lightweight `MemListener` to issue interpretable `ADD`, `UPDATE`, `DELETE`, and `NO_OP` operations. The reported result that a smaller specialized model can perform memory-operation decisions competitively with much larger reasoning models is particularly relevant to the multi-tier compute architecture proposed in Section 12. DOI: `10.18653/v1/2026.acl-long.614`. URL: https://aclanthology.org/2026.acl-long.614/

- **Tian et al. (2026), "SwiftMem: Fast Agentic Memory via Query-aware Indexing."** SwiftMem targets the retrieval-scaling problem using temporal indexing and a hierarchical semantic `DAG-Tag` index, with embedding-tag co-consolidation. The authors report up to `47×` faster search than evaluated baselines while maintaining competitive accuracy on LoCoMo and LongMemEval. This is closely related to the present paper's claim that structured semantic control metadata can reduce the search space before full-memory retrieval. arXiv: `2601.08160`. URL: https://arxiv.org/abs/2601.08160

Contemporary memory systems and temporal organization

- **Yu et al. (2026), "Agentic Memory: Learning Unified Long-Term and Short-Term Memory Management for Large Language Model Agents," ACL 2026.** AgeMem integrates short- and long-term memory operations into the agent policy and learns when to store, retrieve, update, summarize, or discard information. It is important prior art for autonomous decisions over *what and when* to operate on memory. The present proposal differs by externalizing routine relevance control into a persistent metadata plane rather than learning all memory control as policy actions. DOI: `10.18653/v1/2026.acl-long.981`. URL: https://aclanthology.org/2026.acl-long.981/

- **Latimer et al. (2026), "Hindsight: Structured Agent Memory that Retains, Recalls, and Reflects," ACL 2026 Demo.** Hindsight separates world, experience, observation, and opinion networks and combines vector, keyword, graph, and temporal retrieval. It is relevant prior art for structured memory, temporal filtering, and explicit separation of facts from beliefs. The metadata-control proposal can route into such structures but does not claim those structures themselves. URL: https://aclanthology.org/2026.acl-demo.27/

- **Hu et al. (2026), "EverMemOS: A Self-Organizing Memory Operating System for Structured Long-Horizon Reasoning," ACL 2026.** EverMemOS implements a lifecycle from episodic trace formation through semantic consolidation to reconstructive recollection. It further narrows the novelty claim around lifecycle and consolidation: the present proposal focuses instead on a compact operational

control layer that can govern activation without requiring full recollection on every step. DOI: 10.18653/v1/2026.acl-long.2125. URL: https://aclanthology.org/2026.acl-long.2125/

- **Chen et al. (2026), "Beyond Semantic Organization: Memory as Execution State Management for Long-Horizon Agents."** MAGE organizes execution history as a hierarchical state tree with Grow, Compress, Maintain, and Revise operations, reporting both task-success gains and reduced token consumption. This is relevant prior art for state-aware memory control beyond semantic similarity. The present proposal is complementary: its metadata plane is backend-agnostic and targets persistent relevance, obligations, and event-triggered reactivation rather than execution-tree reconstruction. arXiv: 2606.06090. URL: https://arxiv.org/abs/2606.06090
- **Omri et al. (2026), "Agent Memory: Characterization and System Implications of Stateful Long-Horizon Workloads."** This systems characterization separates construction, retrieval, and generation costs across representative memory systems and derives recommendations around construction scheduling, amortization, and freshness/latency tradeoffs. It strengthens the motivation for measuring *where* memory cost is paid rather than only end-task accuracy. arXiv: 2606.06448. URL: https://arxiv.org/abs/2606.06448
- **Xu et al. (2025), "A-MEM: Agentic Memory for LLM Agents."** A-MEM uses LLM-driven note construction, link generation, and memory evolution in a Zettelkasten-inspired organization. It is important prior art for dynamically linked and evolving memory structures; the present proposal does not claim semantic links or memory evolution individually.
- **Rasmussen et al. (2025), "Zep: A Temporal Knowledge Graph Architecture for Agent Memory."** Zep/Graphiti incrementally builds a temporally aware knowledge graph and preserves changing relationships over time. This is direct prior art for temporal validity and historical relationships. The metadata-control proposal uses such information as possible routing/control metadata and is compatible with temporal graph backends rather than replacing them. arXiv: 2501.13956. URL: https://arxiv.org/abs/2501.13956
- **Li et al. (2025), "MemOS: A Memory OS for AI System."** MemOS treats memory as a manageable system resource and introduces `MemCube` objects carrying content together with metadata including provenance and versioning, plus scheduling and evolution across memory types. This substantially overlaps with the broad idea of memory orchestration and metadata-bearing memory objects. The narrower distinction of the present proposal is the use of a compact persistent metadata plane specifically to avoid repeated semantic evaluation during routine relevance control. arXiv: 2507.03724. URL: https://arxiv.org/abs/2507.03724

- **LangMem (LangChain).** LangMem provides hot-path memory tools and a background memory manager that extracts, consolidates, and updates agent knowledge over time. It is practical prior art for separating foreground agent work from background memory maintenance. The present proposal adds an explicit control-plane hypothesis involving persistence classes, triggers/polling, and metadata-first activation. URL: https://github.com/langchain-ai/langmem
- **Wei et al. (2026), "MemLens: A Value-Aware Memory Management System with Interactive Analytics for LLM-based Agents."** MemLens treats memory records as first-class data objects and exposes a complete lifecycle with value-aware storage plus hierarchical visualization and evaluation of latency/token consumption. This is important prior art for lifecycle-aware and value-aware memory management. The present proposal therefore does not claim first-class memory objects, lifecycle management, hierarchy, or value scoring individually; its claim is limited to the specific metadata-first control-plane integration. arXiv: 2607.25992. URL: https://arxiv.org/abs/2607.25992

Proactive memory activation

- **Wu et al. (2026), "Remember When It Matters: Proactive Memory Agent for Long-Horizon Agents."** This work identifies **behavioral state decay**: decision-relevant state can cease to influence an agent as trajectories grow. A separate memory agent maintains structured memory and selectively injects memory-grounded reminders into an otherwise unmodified action agent. The reported gains and ablations provide empirical support for the broader premise that memory should sometimes intervene proactively rather than waiting for passive similarity retrieval. arXiv: 2607.08716. URL: https://arxiv.org/abs/2607.08716
- **Liu and Gabriel (2026), "PM-Bench: Evaluating Prospective Memory in LLM Agents."** PM-Bench evaluates whether agents retain delayed intentions and execute them when future time/event/state cues occur while other tasks continue. The benchmark is directly relevant to the polling/event-trigger portion of this proposal: those mechanisms should be evaluated not only for retrospective recall but also for prospective obligations. arXiv: 2607.12385. URL: https://arxiv.org/abs/2607.12385

Long-horizon memory evaluation and retrieval limitations

- **Wu et al. (2025), "LongMemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory."** LongMemEval tests information extraction, multi-session reasoning, knowledge updates, temporal reasoning, and abstention over timestamped histories. It is a natural baseline for testing whether metadata control improves retrieval under updates and temporal distance. Project:

https://github.com/xiaowu0162/LongMemEval

- **Maharana et al. (2024), "LoCoMo: Evaluating Very Long-Term Conversational Memory of LLM Agents."** LoCoMo evaluates question answering, event summarization, and multimodal dialogue over very long-term conversations and is widely used by later agent-memory systems. It should be included as a standard conversational-memory benchmark alongside task-oriented long-horizon tests. Project: https://snap-research.github.io/locomo/

- **Zhao et al. (2026), "AMA-Bench: Evaluating Long-Horizon Memory for Agentic Applications."** AMA-Bench evaluates recall, causal inference, state updating, and state abstraction over real and synthetic agent trajectories. The authors report that existing memory systems are limited by missing causal/objective information and by lossy similarity-based retrieval; their AMA-Agent combines a causality graph with tool-augmented retrieval. This supports the need for control metadata that preserves objectively important state even when semantic similarity is weak. arXiv: `2602.22769`. URL: https://arxiv.org/abs/2602.22769

- **Lee et al. (2026), "LongMINT: Evaluating Memory under Multi-Target Interference in Long-Horizon Agent Systems."** LongMINT contains 15.6k QA pairs over contexts averaging 138.8k tokens and extending to 1.8M tokens. Across seven representative systems, the authors report an average accuracy of 27.9%, with retrieval and memory construction identified as major limitations and performance degrading under repeated updates and interference. These results motivate explicit lifecycle, supersession, conflict, and persistence metadata. arXiv: `2605.18565`. URL: https://arxiv.org/abs/2605.18565

- **Backlund and Petersson (2025), "Vending-Bench: A Benchmark for Long-Term Coherence of Autonomous Agents."** Vending-Bench evaluates autonomous agents operating a simulated vending-machine business over trajectories exceeding 20M tokens. Reported failures include forgetting orders, misinterpreting delivery schedules, and long-running coherence breakdowns. The recurring-fee scenario used in Section 15 is a deliberately simplified Vending-Bench-style test case rather than a claim that the benchmark implements the exact metadata mechanism proposed here. arXiv: `2502.15840`. URL: https://arxiv.org/abs/2502.15840

Procedural and workflow memory

- **Wang et al. (2024), "Agent Workflow Memory."** AWM induces reusable workflows from prior trajectories and selectively supplies them to agents on later tasks. It is prior art for procedural/experience-derived memory and selective provision of stored structures. The present proposal is not a procedural-memory method, but its control layer could route

to workflow memories as one backend type. arXiv: 2409.07429. URL: https://arxiv.org/abs/2409.07429

Memory consolidation, forgetting, and multi-cue retrieval

- **Keresteciogl u et al. (Microsoft Research, 2026), "Human-Inspired Memory Architecture for LLM Agents."** This architecture combines sleep-phase consolidation, interference-based forgetting, engram maturation, reconsolidation on retrieval, entity knowledge graphs, and hybrid multi-cue retrieval. Its explicit treatment of consolidation and forgetting provides relevant complementary prior work for the lifecycle-management and fallback-retrieval aspects of the present proposal. URL: https://www.microsoft.com/en-us/research/publication/human-inspired-memory-architecture-for-llm-agents/

Additional efficiency and reliability context

- **Xiong et al. (2026), "How Memory Management Impacts LLM Agents: An Empirical Study of Experience-Following Behavior," ACL 2026.** The study shows that memory addition/deletion and similarity-driven replay can propagate errors or replay misleading experiences. This supports conservative deletion, provenance, conflict checking, and fallback validation in Sections 9 and 16. DOI: 10.18653/v1/2026.acl-long.27. URL: https://aclanthology.org/2026.acl-long.27/
- **Dai et al. (2026), "RecMem: Recurrence-based Memory Consolidation for Efficient and Effective Long-Running LLM Agents," Findings of ACL 2026.** RecMem challenges eager LLM-based consolidation of every interaction and instead invokes expensive consolidation when sustained semantic recurrence makes extraction worthwhile. It reports reductions of up to 87% in memory-construction token cost while improving accuracy over evaluated baselines. This is closely related to the present paper's efficiency motivation, but addresses **when to consolidate incoming experience** rather than using a persistent metadata control plane to govern ongoing relevance, persistence, activation, and selective retrieval after classification. DOI: 10.18653/v1/2026.findings-acl.1619. URL: https://aclanthology.org/2026.findings-acl.1619/
- **Zhang et al. (2026), "Useful Memories Become Faulty When Continuously Updated by LLMs."** This work finds that repeated LLM consolidation can corrupt otherwise useful memory and argues that raw episodic traces should remain first-class evidence while consolidation is explicitly gated. This motivates a conservative interpretation of the lifecycle operations in Section 9: metadata may be updated aggressively, but source memories should preferentially be archived, versioned, or soft-deleted rather than destructively rewritten or discarded without retained

provenance. arXiv: 2605.12978. URL: https://arxiv.org/abs/2605.12978

- **DRIFTBENCH (2026), "Long-Horizon Memory Benchmark For AI Agents."** DRIFTBENCH evaluates memory architectures under revisions, distractor accumulation, interference, and temporal decay, and reports stronger results for hybrid/temporal approaches than for simple sliding-window or Vector-RAG baselines. Its findings support evaluating explicit temporal validity, version control, selective retrieval, and interference resistance rather than treating semantic similarity or context length as sufficient measures of persistent-memory quality. URL: https://www.svedbergopen.com/index.php/ijaiml/article/view/592
- **Zhang et al. (2026), "Lightweight LLM Agent Memory with Small Language Models," ACL 2026.** LightMem separates online memory processing from longer-term consolidation and delegates memory operations to smaller language models under bounded compute. It provides additional prior work for the multi-tier compute assumption in Section 12: memory control need not require the same large reasoning model used for the agent's primary task. URL: https://aclanthology.org/2026.acl-long.588/

These works sharpen the scope of the present proposal. The contribution is **not** the general observation that memory processing should be cheaper, delayed, hierarchical, or delegated to smaller models. The narrower hypothesis is that, once semantic interpretation has occurred, a persistent metadata control plane can preserve much of the information needed for routine relevance management and can decide when expensive memory access or semantic re-evaluation is warranted.

Positioning of this proposal

These systems overlap with individual components of the proposed architecture, but the intended contribution here is their combination into a **metadata-first control plane**: semantic interpretation is concentrated around memory-state changes; the resulting tags, weights, persistence classes, triggers, polling schedules, lifecycle state, and relations become first-class control objects; and full semantic retrieval or large-model reasoning is deferred until those cheap control mechanisms identify a reason to reactivate memory.

Accordingly, the paper does **not** claim invention of learned memory managers, memory lifecycles, value/importance weighting, semantic or hierarchical tag indexes, temporal validity/versioning, proactive or prospective memory, workflow memory, background consolidation, memory operating systems, proactive memory injection, or long-horizon memory evaluation individually. The research hypothesis is that integrating these ideas around a persistent metadata control plane can reduce repeated semantic work while improving the survival and timely reactivation of objectively important state.

This literature review is intended to establish technical positioning, **not legal freedom-to-operate or patentability**. A publication-ready technical paper can cite known academic and product prior art, but any patent or freedom-to-operate claim would require a dedicated professional patent search across claims, families, jurisdictions, and unpublished applications that ordinary web research cannot establish.

AI Assistance and Acknowledgements

This whitepaper was developed with AI-assisted research, literature review, structural editing, and drafting support using ChatGPT by OpenAI. The author directed the research questions, architecture, scope, claims, and final editorial decisions. AI assistance is disclosed here for transparency and is not presented as independent scientific authorship.

Publication Status

v1.0 — public technical whitepaper.

This document presents a conceptual architecture and research hypotheses. The literature review establishes technical positioning against known academic and practical systems as of 2026-08-12. Resource-efficiency and recall improvements remain hypotheses until validated by the evaluation proposed in Section 14.